

LENGUAJE PHP

Código Embebido

Los bloques de código se escriben entre `<?php` y `?>`, mientras que las sentencias se separan mediante `;`.

```
<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="UTF-8">
<title>PHP fácil</title>
</head>
<body>
<!-- Muestra una frase con HTML -->
Hola mundo<br>
<!-- Muestra una frase con PHP -->
<?php echo "Es muy fácil programar en PHP."; ?>
</body>
</html>
```

Generando contenido

Tenemos tres posibilidades a la hora de generar contenido en nuestros documentos PHP:

```
echo expresión;
print (expresión);
<?= expresión ?>
```

Las que vamos a utilizar son `echo` cuando lo hagamos dentro de un bloque de instrucciones y `<?=` cuando sólo vayamos a mostrar el valor de una variable dentro de un fragmento HTML.

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Echo y print</title>
</head>

<body>

<p><?php echo "Este texto se mostrará en la página web." ?></p>
<p><?php "Este texto se mostrará en la página web." ?></p>
<p><?php print("Este texto se mostrará en la página web.") ?></p>

</body>
</html>
```

Comentarios

Podemos utilizar comentarios de una línea o de bloque:

```
<?php
// Este es un comentario de una sola línea
/*
    Este es
    un comentario
    que ocupa
    varias líneas
*/
?>
```

Variables

No es necesario declararlas previamente.

Comienzan por \$, por ejemplo \$nombre. Tras el \$, el siguiente caracter debe ser una letra en minúscula (recomendación) o guión bajo _. Luego ya se pueden poner números.

Son case sensitive: \$var != \$vAR

No se declara su tipo, el tipado es dinámico. Se asigna en tiempo de ejecución dependiendo del valor asignado.

Conveniente inicializarlas, sino dan error.

```
<?php
$nombre = "Julio";
$nombreCompleto = "Julio Noguera";
$numero = 123;
$numero2 = 456;
$pi = 3.14;
$suerte = true;
$sinValor;

echo $sinValor;
?>
```

Aunque a priori no hay tipos de datos, internamente PHP trabaja con cuatro tipos escalares: **boolean**, **integer**, **float** y **string** y cuatro tipos compuestos: **array**, **object**, **callable** e **iterable**.

Existe un tipo especial para null (más información en [enlace](#)).

Constantes

Son variables cuyo valor no varían. Existen dos posibilidades:

```
define(NOMBRE, valor);
const NOMBRE; // PHP > 5.3

<?php
define("PI", 3.1416);
const IVA = 0.21;
```

```
echo PI, " ", IVA; // No se pone el símbolo dolar
?>
```

Se declaran siempre en MAYÚSCULAS.

Hay un conjunto de constantes ya predefinidas, también conocidas como magic constants:
<https://www.php.net/manual/es/language.constants.predefined.php>

Operadores

Operadores Ariméticos

Ejemplo	Nombre	Resultado
$\$a + \b	Suma	Suma de \$a y \$b.
$\$a - \b	Resta	Diferencia de \$a y \$b.
$\$a * \b	Multipliación	Producto de \$a y \$b.
$\$a / \b	División	Cociente de \$a y \$b.
$\$a \% \b	Módulo / Resto	Resto de \$a dividido por \$b.
$\$a ** \b	Potencia	Resultado de \$a elevado a \$b. (PHP >= 5.6)

En el caso de cadenas, si queremos concatenarlas, se utiliza el operador '.'

```
<?php
$x = 33;
$y = 11;
$z = $x + $y;

echo "La suma de ".$x." y ".$y." es ".(33 + 11)."<br />";
echo "La suma de ".$x." y ".$y." es ".$z."<br />";
echo "La suma de $x y $y es $z <br />";

?>
```

Realmente, en vez de concatenar cadenas con variables, podemos imprimirlas directamente ya que se expanden automáticamente:

```
<?php
echo "La suma de $x y $y es $z <br />";

?>
```

En ocasiones, necesitamos rodear el nombre de la variable entre llaves para poder unir más texto al resultado:

```
<?php
$color = "rojo";

echo "El plural de $color el ${color}s";
echo "El plural de $color el $color"."s";

?>
```

Más adelante estudiaremos algunas funciones para el tratamiento de cadenas.

Operadores Comparación

Ejemplo	Nombre	Resultado
<code>\$a == \$b</code>	Igual	true si \$a es igual a \$b tras la conversión de tipos.
<code>\$a === \$b</code>	Idéntico, Comparación estricta	true si \$a es igual a \$b y son del mismo tipo de dato. <code>var_dump('5'===5);</code> <code>var_dump('5'==='5');</code>
<code>\$a != \$b, \$a <> \$b</code>	Diferente	true si \$a no es igual a \$b después de la conversión de tipos.
<code>\$a !== \$b</code>	No idéntico	true si \$a no es igual a \$b o si no son del mismo tipo.
<code>\$a < \$b</code>	Menor que	true si \$a es estrictamente menor que \$b.
<code>\$a > \$b</code>	Mayor que	true si \$a es estrictamente mayor que \$b.
<code>\$a <= \$b</code>	Menor o igual que	true si \$a es menor o igual que \$b.
<code>\$a >= \$b</code>	Mayor o igual que	true si \$a es mayor o igual que \$b. Devuelve -1, 0 o 1 cuando \$a es respectivamente menor, igual o mayor que \$b. (PHP >= 7)
<code>\$a <=> \$b</code>	Nave espacial	<code>var_dump('b'<=>'a');</code> <code>var_dump(2<=>7);</code> <code>var_dump(2<=>2);</code> <code>var_dump(7<=>2);</code>

Operadores Lógicos

Ejemplo	Nombre	Resultado
<code>\$a and \$b, \$a && \$b</code>	And Sí	true si tanto \$a como \$b son true.
<code>\$a or \$b, \$a</code>		\$b
<code>!\$a</code>	Not (no)	true si \$a no es true.

Operadores Asignación

Ejemplo	Nombre	Resultado
<code>\$a = \$b</code>	Asignación	Asigna a \$a el valor de \$b.
<code>\$a += \$b</code>	Asignación de la suma	Le suma a \$a el valor de \$b. Equivalente a <code>\$a = \$a + \$b</code> .
<code>\$a -= \$b</code>	Asignación de la resta	Le resta a \$a el valor de \$b. Equivalente a <code>\$a = \$a - \$b</code> .
<code>\$a *= \$b</code>	Asignación del producto	Asigna a \$a el producto de \$a por \$b. Equivalente a <code>\$a = \$a * \$b</code> .
<code>\$a /= \$b</code>	Asignación de la división	Asigna a \$a el cociente de \$a entre \$b. Equivalente a <code>\$a = \$a / \$b</code> .
<code>\$a %= \$b</code>	Asignación del resto	Asigna a \$a el resto de dividir \$a entre \$b. Equivalente a <code>\$a = \$a % \$b</code> .
<code>\$a .= \$b</code>	Concatenación	Concatena a \$a la cadena \$b. Equivalente a <code>\$a = \$a . \$b</code> .
<code>\$a++</code>	Incremento	Incrementa \$a en una unidad. Equivalente a <code>\$a = \$a + 1</code> .
<code>\$a--</code>	Decremento	Decrementa \$a en una unidad. Equivalente a <code>\$a = \$a - 1</code> .

Condiciones

La condición simple se realiza mediante la instrucción `if`. Entre paréntesis se pone la condición que se evalúa a `true` o `false`. Si no se ponen llaves, en vez de abrir un bloque, se ejecutará sólo la siguiente instrucción.

Siempre llaves

Es recomendable poner llaves siempre aunque en el momento de codificar sólo haya una única instrucción. De este modo, se queda preparado para añadir más contenido en el futuro sin provocar bugs.

```
<?php
$hora = 8; // La hora en formato de 24 horas

if ($hora === 8) {
    echo "Suenas el despertador.";
}

echo "<br>";

if ($hora === 8)
    echo "Suenas el despertador.";
?>
```

Las condiciones compuesta mediante if-else:

```
<?php
$hora = 17; // La hora en formato de 24 horas

if ($hora <= 12) {
    echo "Son las " . $hora . " de la mañana";
} else {
    echo "Son las " . ($hora - 12) . " de la tarde";
}
?>
```

Las condiciones anidadas mediante if-else if-else:

```
<?php

$hora = 14; // La hora en formato de 24 horas

if ($hora === 8) {
    echo "Es la hora de desayunar.";
} else if ($hora === 14) {
    echo "Es la hora de la comida.";
} else if ($hora === 21) {
    echo "Es la hora de la cena.";
} else {
    echo "Ahora no toca comer.";
}
?>
```

La sentencia switch también permite trabajar con condiciones múltiples:

```
<?php
$hora = 14; // La hora en formato de 24 horas

switch ($hora) {
    case 9:
```

```

        echo "Es la hora de desayunar.";
        break;
    case 14:
        echo "Es la hora de la comida.";
        break;
    case 21:
        echo "Es la hora de la cena.";
        break;
    default:
        echo "Ahora no toca comer";
}
?>

```

Finalmente, también tenemos el operador ternario condición ? valorTrue : valorFalse:

```

<?php

$hora = 14;

$formato = ($hora > 12) ? 24 : 12;

echo "El formato es de $formato horas"
?>

```

Bucles

Mediante la instrucción *WHILE*:

```

<?php
$i = 1;
while ($i <= 10) {
    echo "Línea " . $i;
    echo "<br>";
    $i++;
}
?>

```

Mediante la instrucción *DO-WHILE*:

```

<?php

do {
    $dado = rand(1, 6);
    // rand() devuelve un valor aleatorio
    echo "Tirando el dado... ";
    echo "ha salido un " . $dado . ".";
    echo "<br>";

} while ($dado != 5);

echo "¡Bien! Saco una ficha de casa.";
?>

```

Mediante la instrucción *FOR*:

```

<?php
// Bucle ascendente
for ($i = 1; $i <= 10; $i++) {
    echo "Línea " . $i;
    echo "<br>";
}

// Bucle descendente
for ($i = 10; $i >= 0; $i--) {
    echo "Línea " . $i;
    echo "<br>";
}
?>

```

Arrays

Arrays

Para almacenar datos compuestos, podemos utilizar tanto arrays sencillos como arrays asociativos (similares a un mapa). En realidad todos los arrays son mapas ordenados compuestos de pares clave-valor.

Del mismo modo que Java, se definen mediante corchetes, son 0-index, y se puede asignar un valor a un posición determinada:

```

<?php
$frutas = array("naranja", "pera", "manzana");

$frutas2 = ["naranja", "pera", "manzana"];

$frutas3 = [];
$frutas3[0] = "naranja";
$frutas3[1] = "pera";
$frutas3[] = "manzana"; // lo añade al final

```

Podemos obtener el tamaño del array mediante la función *count(array)*. Para recorrer el array haremos uso de un bucle for:

```

<?php
$tam = count($frutas); // tamaño del array

for ($i=0; $i<count($frutas); $i++) {
    echo "Elemento $i: $frutas[$i] <br />";
}

```

Otra forma de recorrer los arrays, incluso más elegante, es hacer uso de foreach. Su sintaxis es foreach (array as elemento):

```

<?php
// Mediante foreach no necesitamos saber el tamaño del array

foreach ($frutas as $fruta) {
    echo "$fruta <br />";
}

```

```
foreach ($frutas as $clave=>$valor) {
    echo "clave: $clave valor: $valor <br />";
}
```

Arrays asociativos

Cada elemento es un par clave-valor. En vez de acceder por la posición, lo hacemos mediante una clave. Así pues, para cada clave se almacena un valor.

A la hora de recorrer este tipo de arrays, mediante foreach separamos cada elemento en una pareja clave => valor:

```
<?php
```

```
$miArray = array('mes1' => 'Enero', 'mes2' => 'Febrero', 'mes3' => 'Marzo');
$miArray1= ['mes1' => 'Enero', 'mes2' => 'Febrero', 'mes3' => 'Marzo'];
```

```
print_r($miArray);
echo "<br>";
print_r($miArray1);
```

```
?>
```

```
<?php
```

```
$array = ["marca"=>"Toyota", "modelo"=>"Celica", "color"=>"black", "fabrica"=>"1991"];
```

```
echo "el coche es un". $array["marca"]. "<br>";
echo "el coche es un {"$array["marca"]} <br>";
```

```
$array['marca']="FERRARI";
$array[]="SPIDER"; //añade con índice [0]
$array['color']="VERDE";
$array[]="2013";
$array['electrico'] = "urbano"; //añade con nueva clave 'electrico' y valor 'urbano'
```

```
print_r($array);
?>
```

Operaciones

Las operaciones más importantes que podemos realizar con arrays son:

Función	Descripción
<code>print_r(\$array)</code>	Muestra el contenido de todo el \$array. Si deseamos mostrar el contenido con un formato determinado, debemos recorrer el array con <code>foreach</code> .
<code>var_dump(\$mixed)</code>	Muestra el contenido del elemento recibido. Proporciona más información que <code>print_r</code> .
<code>\$elem = array_pop(\$array)</code>	Elimina el último elemento del \$array.

Función	Descripción
<code>array_push(\$array, \$elem)</code>	Añade un elemento al final del \$array.
<code>array_splice(\$array, 1, 0, \$elemento)</code>	Inserta el \$elemento (valor) en la posición 1 del \$array.
<code>\$booleano = in_array(\$elem, \$array)</code>	Averigua si \$elem está en el \$array.

```

<?php

$frutas = ["naranja", "pera", "manzana"];

array_push($frutas, "piña");
print_r($frutas);

$sultFruta = array_pop($frutas);

if (in_array("piña", $frutas)) {
    echo "<p>Queda piña</p>";
} else {
    echo "<p>No queda piña</p>";
}
print_r($frutas);

```

Función	Descripción
<code>\$claves = array_keys(\$array)</code>	Devuelve las claves del \$array asociativo.
<code>\$tam = count(\$array)</code>	Devuelve el tamaño de \$array.
<code>sort(\$array)</code>	Ordena los elementos de \$array.
<code>unset(\$array[elemento])</code>	Elimina el elemento del array (deja un hueco). Por ejemplo: <code>unset(\$miArray[1]);</code>

```

<?php
$capitales = array("Italia" => "Roma", "Francia" => "Paris", "Portugal" => "Lisboa");

$países = array_keys($capitales);
print_r($países);

sort($países);
print_r($países);

unset($capitales["Francia"]);
print_r($capitales);

```

Al asignar un array a otro se realiza una copia. Cuidado con esta operación que puede consumir muchos recursos.

```

<?php
$nombres = ["Juan", "Ana", "Pedro", "Laura"];
$copia = $nombres;

sort($nombres);

```

```
print_r($nombres);  
print_r($copia);
```

Existen muchísimas más funciones para trabajar con arrays. Puedes consultar toda la información en la documentación oficial.

Array Bidimensional

```
$cars = array(  
    "car1" => array("make" => "Toyota", "colour" => "Green", "year" => 1999, "engine_cc" => 1998),  
    "car2" => array("make" => "BMW", "colour" => "RED", "year" => 2005, "engine_cc" => 2400),  
    "car3" => array("make" => "Renault", "colour" => "White", "year" => 1993, "engine_cc" => 1395),  
);  
?>
```

```
<?php  
echo $cars['car1']['make'], "<br>";  
echo $cars['car3']['engine_cc'];
```

Combinando los arrays asociativos en varias dimensiones podemos almacenar la información como si fuera una tabla:

```
<?php  
$menu1 = ["Plato1" => "Macarrones con queso", "Plato2" => "Pescado asado", "Bebida" => "Coca-Cola", "Postre" => "Helado"];  
$menu2 = ["Plato1" => "Sopa", "Plato2" => "Lomo con patatas", "Bebida" => "Agua", "Postre" => "Arroz con leche"];  
  
$menus = [$menu1, $menu2]; // creamos un array a partir de arrays asociativos
```

```
foreach ($menus as $menudeldia) {  
    echo "Menú del día<br/>";  
  
    foreach ($menudeldia as $platos => $comida) {  
        echo "$platos: $comida <br/>";  
    }  
}  
  
// Para acceder a un elemento concreto se anidan los corchetes  
$postre0 = $menus[0]["Postre"];
```

Funciones de arrays

Las funciones más usadas en el manejo de arrays en PHP son:

Creación y Extracción

- ⑩ `array` — Crea un array
- ⑩ `extract` — Importar variables a la tabla de símbolos actual desde un array
- ⑩ `list` — Asigna variables como si fuera un array

Recorrido de elementos

- ⑩ `next` — Avanza el puntero interno de un array

- ⑩ `pos` — Alias de `current`
- ⑩ `prev` — Rebobina el puntero interno del array
- ⑩ `end` — Establece el puntero interno de un array a su último elemento
- ⑩ `count` — Cuenta todos los elementos de un array o en un objeto
- ⑩ `current` — Devuelve el elemento actual en un array
- ⑩ `each` — Devolver el par clave/valor actual de un array y avanzar el cursor del array
- ⑩ `key` — Obtiene una clave de un array
- ⑩ `array_walk` — Aplicar una función de usuario a cada miembro de un array

Búsqueda

- ⑩ `in_array` — Comprueba si un valor existe en un array usando comparación flexible
- ⑩ `array_key_exists` — Verifica si el índice o clave dada existe en el array
- ⑩ `key_exists` — Alias de `array_key_exists`
- ⑩ `array_search` — Busca un valor determinado en un array y devuelve la clave correspondiente en caso de éxito

Ordenación

- ⑩ `sort` — Ordena un array
- ⑩ `rsort` — Ordena un array en orden inverso
- ⑩ `uasort` — Ordena un array con una función de comparación definida por el usuario y mantiene la asociación de índices
- ⑩ `uksort` — Ordena un array según sus claves usando una función de comparación definida por el usuario
- ⑩ `usort` — Ordena un array según sus valores usando una función de comparación definida por el usuario
- ⑩ `krsort` — Ordena un array por clave en orden inverso
- ⑩ `ksort` — Ordena un array por clave
- ⑩ `natsort` — Ordena un array usando un algoritmo de "orden natural"

Operaciones Varias

- ⑩ `array_column` — Devuelve los valores de una única columna del array de entrada
- ⑩ `array_keys` — Devuelve todas las claves de un array o un subconjunto de claves de un array
- ⑩ `array_values` — Devuelve todos los valores de un array
- ⑩ `array_combine` — Crea un nuevo array, usando una matriz para las claves y otra para sus valores
- ⑩ `array_count_values` — Cuenta todos los valores de un array
- ⑩ `array_diff` — Calcula la diferencia entre arrays
- ⑩ `array_intersect` — Calcula la intersección de arrays
- ⑩ `array_map` — Aplica la llamada de retorno especificada a los elementos de cada array
- ⑩ `array_merge` — Combina dos o más arrays
- ⑩ `array_pop` — Extrae el último elemento del final del array

- ⑩ `array_push` — Inserta uno o más elementos al final de un array
- ⑩ `array_replace_recursive` — Reemplaza los elementos de los arrays pasados al primer array de forma recursiva
- ⑩ `array_replace` — Reemplaza los elementos de los arrays pasados en el primer array
- ⑩ `array_sum` — Calcula la suma de los valores en un array
- ⑩ `array_unique` — Elimina valores duplicados de un array
- ⑩ `range` — Crear un array que contiene un rango de elementos
- ⑩ `shuffle` — Mezcla un array

Pulsa aquí para acceder a la referencia completa de funciones de arrays.

-